eBPF Edge Stealth Gate

Ivan Sen
Lviv, Ukraine
sen.pn@ucu.edu.ua

Maksym Dzoban
Lviv, Ukraine
dzoban.pn@ucu.edu.ua

Maksym Bilyk
Lviv, Ukraine
m.bilyk@ucu.edu.ua

***Abstract*—This project presents an eBPF-based secure port knocking system designed to hide protected network services from unauthorized clients. The proposed approach uses an XDP program attached at the network interface to inspect incoming IPv4/TCP packets before they reach the host networking stack. A client must first send a specially crafted knock packet containing a keyed HMAC-SHA256 signature over selected packet metadata. If the signature is valid for a registered user, the client source IP is temporarily authorized to access configured protected service ports, such as SSH or HTTPS. Unauthorized traffic to protected ports is dropped by default, making the services appear closed or unreachable.**

***Index Terms*—eBPF, XDP, port knocking, HMAC, access control, Linux networking, packet filtering, service stealth**

I. Introduction

Public network services such as SSH and HTTPS are commonly exposed to the Internet and can be discovered through automated scanning. Even when these services use strong authentication, their visibility increases the attack surface by allowing attackers to identify targets, fingerprint software, and attempt brute-force or vulnerability-based attacks.

This project proposes an eBPF-based edge stealth gate that hides selected services from unauthorized clients. The proposed system uses an XDP program attached to a network interface to inspect packets before they reach the host network stack. A client must first send an authenticated knock packet before access to protected ports is temporarily allowed.

II. Background

A. eBPF and XDP

Extended Berkeley Packet Filter (eBPF) is a Linux kernel mechanism that allows small, verified programs to run inside the kernel without modifying kernel source code or loading a custom kernel module. eBPF programs can be attached to different kernel hooks, including networking, tracing, and security-related subsystems. Before execution, the kernel verifier checks that the program satisfies safety constraints, such as bounded memory access and valid control flow. This makes eBPF suitable for extending kernel behavior while reducing the risk normally associated with running custom code in kernel space [1], [2].

The eXpress Data Path (XDP) is an eBPF-based networking hook placed very early in the packet receive path. An XDP program is executed when an incoming packet arrives at a network interface, before the packet is passed to the regular Linux network stack. At this stage, the program can make simple but efficient decisions, such as allowing the packet to continue, dropping it, redirecting it, or transmitting it back out of the interface [3]. Because XDP operates before socket processing and before most higher-level firewall logic, it is well suited for filtering unwanted traffic close to the edge of the host.

In this project, XDP is used as the enforcement point for the stealth gate. The XDP program inspects incoming IPv4/TCP packets and decides whether they should be passed to the host network stack or dropped. Packets targeting protected ports are dropped unless the source address has previously been authorized by a valid knock packet. This approach keeps the protected service hidden from unauthorized clients while avoiding the need to expose the service directly to all incoming traffic.

B. Port Knocking

Port knocking is an access-control technique in which a protected service remains unreachable until a client sends a predefined signal to the server. In traditional implementations, this signal is often represented as a sequence of connection attempts to closed ports. A daemon observes the sequence and, if it matches the expected pattern, temporarily modifies firewall rules to allow the client to access a protected service. This makes services such as SSH less visible to automated scanning because the service port appears closed until a valid knock is received [4].

Although port knocking can reduce service exposure, simple sequence-based schemes have important limitations. A static port sequence can be observed and replayed by an attacker with access to the network path. Traditional implementations may also depend on userspace daemons and firewall rule updates, which can introduce delay and operational complexity. In addition, basic sequence-based knocking usually provides weak identity management, since the knock often proves knowledge of a shared sequence rather than authenticating a specific registered user.

The system proposed in this project follows the same general idea of hiding a service until a client proves authorization, but replaces static port sequences with a signed single-packet knock. Instead of opening access after observing a fixed sequence, the stealth gate validates authentication data carried inside the knock packet. If the packet is valid, the client source IP address is added to a temporary authorization map. This keeps the port knocking model while making it better suited for per-user keys, key rotation, and revocation.

C. Message Authentication Codes

A Message Authentication Code (MAC) is used to verify that a message was created by a party that knows a shared secret key and that the message was not modified after it was generated. Hash-based Message Authentication Code (HMAC) is a MAC construction based on a cryptographic hash function and a secret key. HMAC is specified in RFC 2104 and standardized by NIST in FIPS 198-1 [5], [6]. Unlike encryption, HMAC does not hide the message contents; instead, it provides authenticity and integrity for the data that is signed.

In the stealth gate, HMAC-SHA256 is used to authenticate knock packets. The client constructs a knock message containing selected metadata, such as the user identifier, timestamp or validity window, session information, and requested access parameters. The client then computes an HMAC over this data using the user's secret key. On the server side, the XDP program or associated userspace logic validates the received authentication value against the stored key for the registered user.

Using HMAC for knock authentication improves over simple port sequences because the authorization signal is bound to packet contents and a secret key. An attacker who does not know the key cannot easily forge a valid knock packet. Including time-based or session-specific fields also helps limit replay attacks, because an observed knock should only remain valid within a restricted window. As a result, the knock packet acts as a compact authentication request rather than merely a hidden trigger.

III. System Model and Threat Assumptions

A. System Model

The proposed system protects selected TCP services running on a Linux host. The host runs an XDP-based eBPF program attached to a network interface. This program acts as an early packet-processing layer for incoming traffic and decides whether packets should be passed to the normal Linux network stack or dropped before they reach the protected service.

The system includes four main actors. The first actor is the protected host, which provides services such as SSH or HTTPS. These services continue to run normally, but traffic to their ports is filtered by the stealth gate. The second actor is an authorized client, which has a registered user identifier and a secret key used to generate authenticated knock packets. The third actor is an unauthorized client or attacker, which may attempt to scan the host, connect directly to protected services, replay observed knocks, or send malformed knock packets. The fourth actor is the administrator, who configures protected ports, registers users, rotates keys, revokes access, and manages the userspace daemon.

Access control is based on a temporary session model rather than a simple source-IP allowlist. By default, packets sent to protected ports are dropped. Before accessing a protected service, a client first sends an authenticated knock packet. If this packet is valid, the system creates a pending authorization associated with the client's source address and session identifier. The client then binds that authorization to a specific TCP flow. The active session is indexed by a tuple containing the source IP address, destination IP address, source port, destination port, and transport protocol. Only packets matching this authorized flow tuple are allowed to pass.

This design prevents a valid knock from opening all protected ports for all traffic from the same source address. Instead, authorization is limited to the specific bound flow and expires after a configured timeout unless it is renewed. For long-running connections, such as SSH sessions, the client can send renew packets to refresh the session before expiration.

B. Threat Model

The attacker is assumed to have network access to the protected host. The attacker can send arbitrary IPv4/TCP packets, attempt to connect to protected ports, perform port scans, replay previously observed traffic, and generate malformed or random knock packets. The attacker may also attempt to discover whether services such as SSH or HTTPS are running on the host.

The attacker is not assumed to know the secret key of a registered user. Without this key, the attacker should not be able to generate a valid authenticated knock packet. The attacker is also not assumed to have administrative access to the protected host, the userspace daemon, the pinned eBPF maps, or the protected service itself. The Linux kernel, the loaded XDP program, and the administrator are treated as trusted components.

A stronger attacker may be able to observe traffic between an authorized client and the protected host. In this case, replay protection is required. The system therefore includes freshness fields, such as timestamps, nonces, packet type, and session identifiers, in the signed knock data. A replay cache is used to reject repeated knock messages within the configured replay window. This limits the usefulness of captured knock packets, although it does not remove the need for secure transport and application-layer authentication after access is granted.

The system does not assume that source IP addresses uniquely identify users. Networks using NAT may cause multiple clients to share the same public source address. The active authorization tuple reduces the impact of this issue by binding access to a specific flow, including source port, destination port, and protocol. However, NAT behavior can still complicate identity and session semantics, so this system should not be treated as a complete identity-management solution.

C. Security Goals

The first security goal is service hiding. Protected services should not appear open to clients that have not completed a valid knock and bind sequence. Direct connection attempts to protected ports should be dropped before they reach the host network stack or the protected application.

The second goal is authenticated access. A knock packet should only be accepted if it was generated by a registered client using the correct secret key. This prevents unauthorized

clients from opening access by guessing the knock format or sending traffic to the knock port.

The third goal is replay resistance. A captured knock packet should not remain valid indefinitely or be reusable without restriction. The system should bind knock validation to freshness data such as timestamps, nonces, packet type, and session identifiers, and it should reject repeated values within the replay window.

The fourth goal is flow-bound temporary authorization. A valid knock should not permanently open a protected service, nor should it authorize all traffic from a source address. Instead, access should be granted only to a specific bound flow tuple and only for a limited time. The authorization should expire automatically unless renewed by the client.

The fifth goal is operational manageability. The administrator should be able to register users, rotate keys, revoke users, and inspect the current configuration without rebuilding the kernel program or restarting the entire host. This allows the stealth gate to support multiple users and key lifecycle operations while keeping the packet-filtering path inside XDP.

D. Non-Goals

This system is not intended to replace application-layer authentication. Services such as SSH and HTTPS must still use their own authentication and encryption mechanisms. The stealth gate reduces service exposure, but it should be treated as an additional access-control layer rather than the only security mechanism.

The system is not designed to fully prevent denial-of-service attacks. Since the XDP program can drop unauthorized packets early, it may reduce the amount of traffic reaching the host network stack. However, it cannot prevent link saturation, large-scale packet floods, or attacks against infrastructure outside the protected host.

The system does not protect against compromise of the protected host, the Linux kernel, the userspace daemon, or registered client keys. If an attacker gains administrative control over the host or obtains a valid user's secret key, the stealth gate cannot guarantee confidentiality or access isolation.

Finally, the system is not intended to solve all identity and access-management problems. Authorization is based on valid knock authentication, session binding, and temporary flow-level allowance. More advanced policies, such as per-connection cryptographic identity, hardware-backed keys, or integration with centralized identity providers, are left for future work.

IV. Proposed Architecture

A. High-Level Design

The proposed system is divided into a kernel-resident fast path and a userspace control plane. The kernel fast path is implemented as an XDP program attached to a selected network interface. It inspects incoming Ethernet, IPv4, and TCP packets before they reach the normal Linux network stack. Its main responsibility is to decide whether a packet should be passed to the host or dropped at the edge.

The userspace control plane consists of a daemon, loader, administrative commands, and a knock client. The daemon loads the eBPF object, configures the protected ports, inserts user keys into eBPF maps, sets timing parameters, and attaches the XDP program to the interface. Administrative commands are used to register users, rotate user keys, revoke users, and list currently configured users. The client utility constructs signed knock packets and sends them using a raw packet socket.

The system does not authorize traffic using only the client source IP address. Instead, it uses a two-stage authorization model. First, a valid authentication knock creates a temporary pending authorization indexed by the client source IP address and session identifier. Second, a bind knock converts that pending authorization into an active session bound to a specific flow tuple. The active session key contains the source IP address, destination IP address, source port, destination port, and transport protocol. Therefore, the final authorization applies to a specific protected connection rather than to every packet from the same host.

The knock protocol supports four packet types: authentication, bind, renewal, and deauthentication. An authentication packet proves that the client knows a registered user's key. A bind packet associates the authenticated session with a concrete protected flow. A renewal packet extends the lifetime of an existing active session. A deauthentication packet removes an active session before its normal timeout expires.

B. Packet Processing Pipeline

Packet processing begins when a frame reaches the XDP hook. The program first checks whether the frame contains an IPv4 packet and whether the transport protocol is TCP. Packets that do not match this format are passed without further processing. For TCP packets, the program extracts the source and destination ports, loads the current configuration from the configuration map, and constructs a flow key from the packet headers.

If the packet is addressed to the configured knock port, it is treated as a control packet. The program parses the embedded knock packet, verifies its magic value, timestamp, nonce, packet type, session identifier, optional bind ports, and signature. Replay protection is enforced using a nonce map, while source-pressure checks limit the number of knock attempts and active sessions associated with one source address.

For an authentication packet, the program checks the user key and inserts a temporary pending authorization into the pending authorization map. This state is indexed by source IP address and session identifier. It does not yet allow traffic to a protected service. For a bind packet, the program checks whether a matching pending authorization exists and has not expired. If the bind is valid, the program creates an active session indexed by the full flow tuple and also creates a reverse session index from the session identifier to the flow key.

For normal service traffic, the program checks whether the destination port is listed as protected. If the port is not protected, the packet is passed. If the port is protected, the

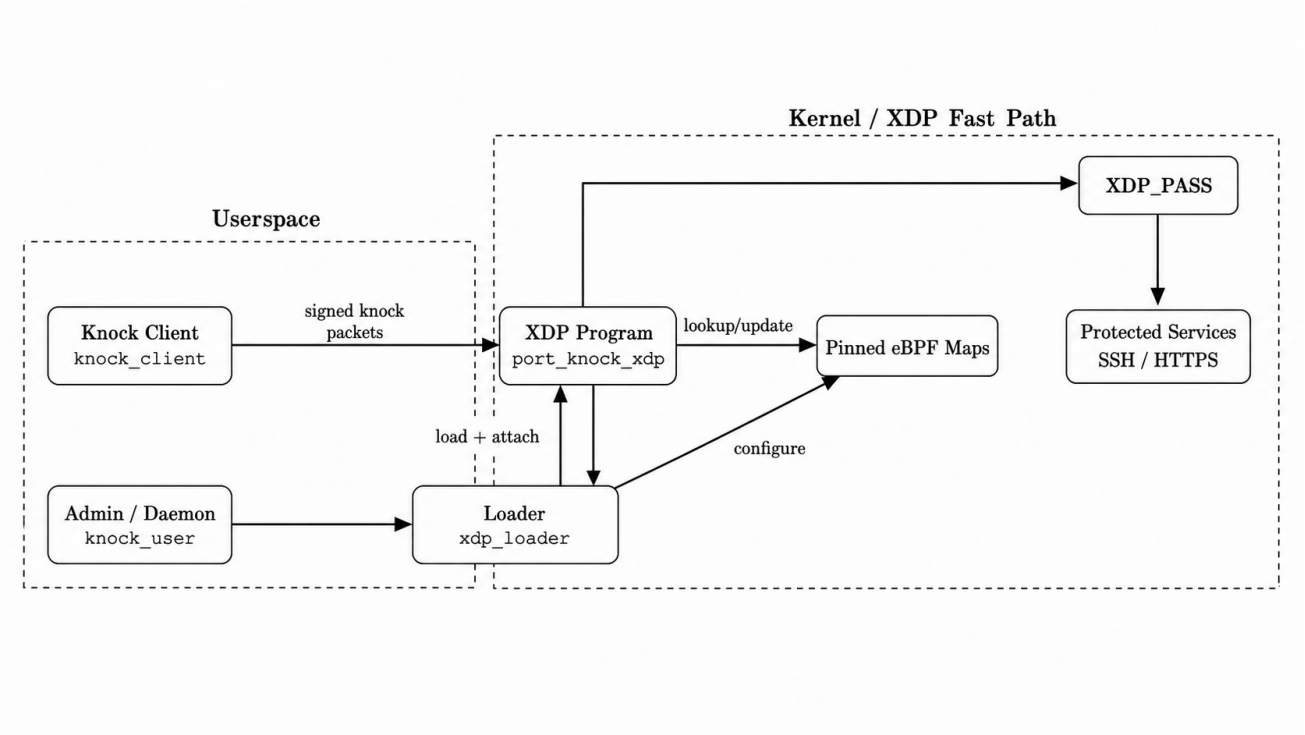

Fig. 1. High-level architecture of the eBPF stealth gate. Userspace loads and configures the XDP program, while packet filtering and session enforcement occur inside the kernel fast path.

program looks up the full flow tuple in the active session map. A packet is passed only when a matching active session exists and has not expired. Otherwise, the packet is dropped. Expired sessions are removed from both the active session map and the reverse session index.

Renewal and deauthentication packets use the reverse session index. A renewal packet resolves the session identifier to the active flow and extends the expiration timestamp. A deauthentication packet resolves the same index and removes the corresponding active session. All knock packets are dropped after processing, even if valid, so that the knock port itself does not expose service behavior to the host network stack.

C. eBPF Maps

The implementation uses eBPF maps to share configuration and runtime state between the userspace control plane and the XDP fast path. The configuration map stores the knock port, protected service ports, session timeout, bind window, replay window, and fallback key material. The loader writes this configuration before attaching the XDP program.

The authorization state is split across three maps. The pending authorization map stores short-lived authentication state created by valid authentication knocks. The active session map stores the actual permission used to pass protected packets. Its key is the full flow tuple, not merely the source IP address. The session index map provides a reverse lookup from source IP address and session identifier to the active flow key. This reverse index is used by renewal and deauthentication packets.

Replay protection is implemented using a replay nonce map. Each entry records a recent nonce together with the packet type and session identifier, preventing simple reuse of captured knock packets inside the replay window. Source pressure is tracked with a source-pressure map, which stores the number of knock attempts and active sessions associated with a source address. This limits both repeated knock attempts and excessive active sessions from the same source.

User authentication keys are stored in the user key map. Each user record contains an active key, a previous key, a key version, and a grace expiration time. This allows key rotation while optionally accepting the previous key during a limited grace period. The implementation also includes maps for debug counters, debug snapshots, realtime offset storage, and per-CPU scratch space used during packet parsing and signature verification.

D. Userspace Components

The main userspace entry point is `knock_user`. In daemon mode, it accepts the interface name, BPF object path, knock port, protected port list, session timeout, bind window, replay window, and user key source. Users can be loaded from a file, or a single fallback user can be created from a provided HMAC key. After parsing and validating these parameters, the daemon calls the loader to initialize and attach the XDP program.

The administrative subcommands operate on pinned maps after the daemon has created them. The `register-user` command inserts a new user key into the user key map.

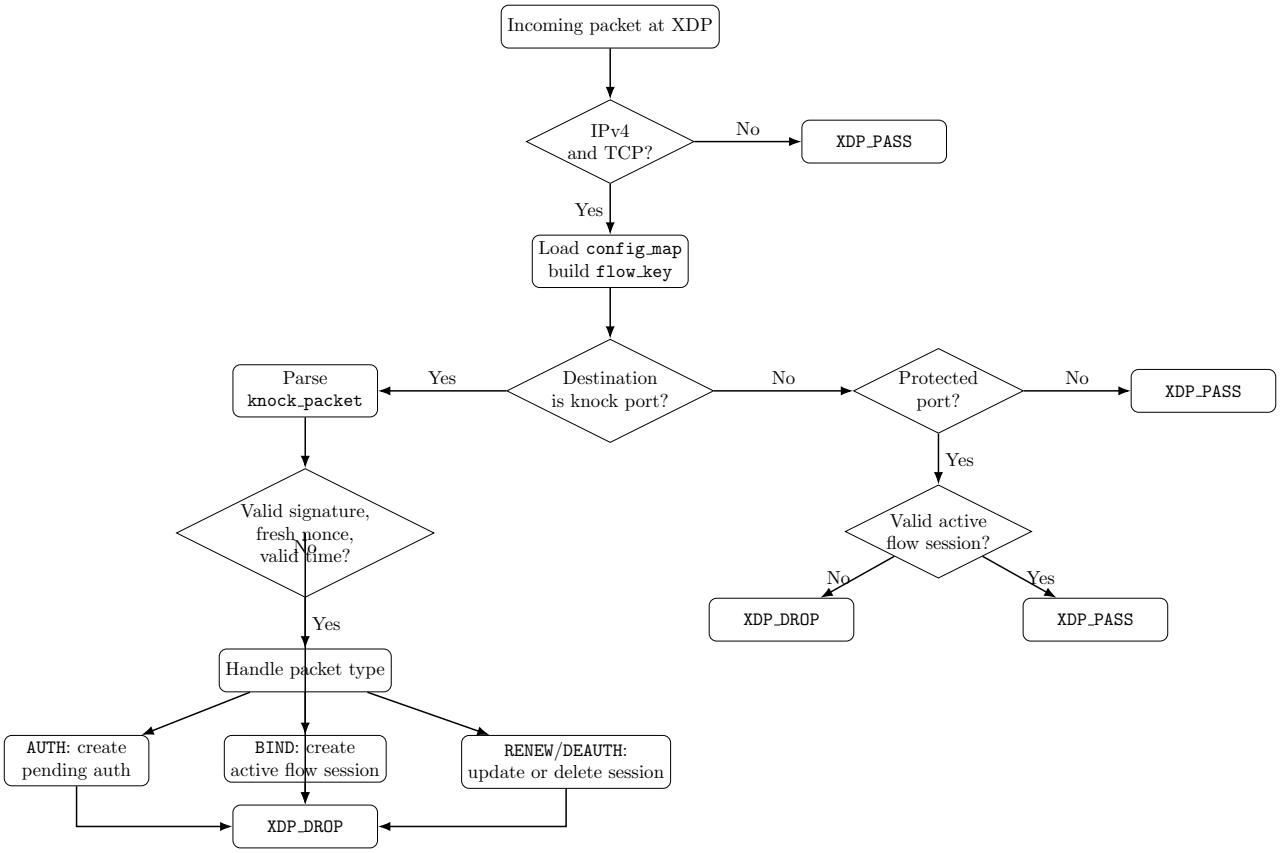

Fig. 2. Packet-processing pipeline used by the XDP program. Knock packets update control state, while ordinary packets to protected ports are checked against active flow sessions.

The `rotate-user-key` command replaces the active key, moves the old key into the previous-key field, increments the key version, and sets a grace period. The `revoke-user` command removes a user from the map, and `list-users` prints the configured user identifiers and key versions.

The loader component is responsible for the low-level eBPF setup. It opens and loads the compiled BPF object, writes the global configuration into `config_map`, computes the realtime-to-monotonic clock offset and stores it in `time_offset_map`, inserts users into `user_key_map`, finds the `port_knock_xdp` program, attaches it to the selected interface, and pins maps under the configured BPF filesystem directory.

The `knock_client` utility constructs control packets. It supports authentication, bind, renewal, and deauthentication packet types. If a nonce or session identifier is not provided, it generates one. For authentication packets, it embeds the user identifier into the session identifier. For bind packets, it includes the source port and protected destination port in the signed input. Finally, it builds an Ethernet, IPv4, TCP, and knock-payload frame and transmits it through a raw packet socket.

V. Implementation

A. Kernel-Side XDP Program

The kernel-side component is implemented as an XDP program named `port_knock_xdp`. The program is attached to a network interface and is executed for incoming packets before they reach the normal Linux networking stack. Its first responsibility is packet classification. It parses the Ethernet header, checks whether the packet is IPv4, verifies that the transport protocol is TCP, and validates the IP and TCP header lengths before accessing further packet data. Non-IPv4 or non-TCP packets are passed without applying the stealth gate policy.

For TCP packets, the program loads the runtime configuration from `config_map`. The configuration contains the knock port, protected port list, session timeout, bind window, and replay window. The program then extracts the TCP source and destination ports and constructs a `flow_key` from the source IP address, destination IP address, source port, destination port, and transport protocol. This flow key is later used for active session lookup.

If the destination port matches the configured knock port, the packet is treated as a control packet. The knock payload is parsed as a `knock_packet`, which contains a magic value,

TABLE I
MAIN EBPF MAPS USED BY THE STEALTH GATE

Map	Key	Value	Purpose
`config_map`	`__u32`	`knock_config`	Stores knock port, protected ports, and timeout configuration.
`pending_auth_map`	`session_lookup_key`	`pending_auth_state`	Stores short-lived authenticated-but-not-bound sessions.
`active_session_map`	`flow_key`	`active_session_state`	Stores actual flow-level authorization for protected traffic.
`session_index_map`	`session_lookup_key`	`flow_key`	Maps a session identifier back to its authorized flow.
`replay_nonce_map`	`replay_nonce_key`	`replay_nonce_state`	Tracks recent nonces for replay protection.
`source_pressure_map`	Source IPv4 address	`source_pressure_state`	Tracks knock rate and active session count per source.
`user_key_map`	User ID	`user_key_state`	Stores active and previous user keys for authentication and rotation.
`stats_map`	`__u32`	`debug_counters`	Stores runtime counters for accepted, dropped, replayed, and invalid packets.
`time_offset_map`	`__u32`	`time_offset_state`	Stores realtime-to-monotonic clock offset for timestamp validation.
`debug_knock_map`	`__u32`	`debug_knock_snapshot`	Stores the last observed knock fields for debugging.
`crypto_scratch_map`	`__u32`	Signature scratch state	Provides per-CPU scratch space for signature computation.
`knock_path_scratch_map`	`__u32`	Knock path scratch state	Provides per-CPU temporary state for packet-path processing.

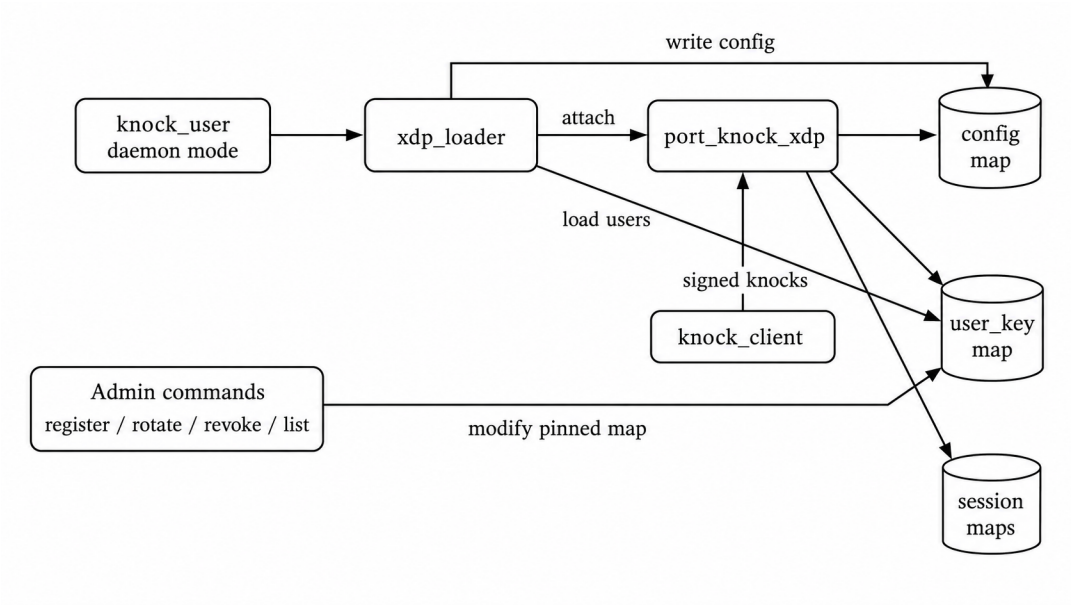

Fig. 3. Userspace responsibilities. The daemon and loader configure the kernel fast path, administrative commands update pinned maps, and the client sends signed control packets.

timestamp, nonce, packet type, session identifier, optional bind ports, and signature words. The XDP program validates the packet format, checks the timestamp against the configured clock-skew window, resolves the user identifier from the high bits of the session identifier, and verifies the signature against the corresponding entry in `user_key_map`. The signature check supports both the active key and, during a configured grace period, the previous key.

Replay protection is implemented using `replay_nonce_map`. The replay key includes the source IP address, nonce, packet type, and session identifier. If the same replay key is already present and has not expired, the knock is dropped. Otherwise, the replay entry is inserted with an expiration timestamp. The program also uses `source_pressure_map` to track knock attempts and active sessions per source address, limiting repeated knock attempts and excessive concurrent sessions.

The knock control path supports four packet types. For an `AUTH` packet, the program creates a `pending_auth_state` in `pending_auth_map`. This pending state is keyed by `session_lookup_key`, which consists of the source IP address and session identifier. The pending state expires after the configured bind window and does not by itself allow protected service traffic.

For a `BIND` packet, the program first checks that a matching pending authorization exists and has not expired. If the pending authorization is valid, the program creates an `active_session_state` in `active_session_map`.

Unlike the pending authorization, the active session is keyed by a full `flow_key`. This means that the final authorization is bound to a specific source IP address, destination IP address, source port, destination port, and transport protocol. The program also creates an entry in `session_index_map`, mapping the session identifier back to the authorized flow. This reverse index is used later for renewal and deauthentication.

For a `RENEW` packet, the program uses `session_index_map` to resolve the session identifier into the active flow key. If the active session still exists and has not expired, its expiration timestamp is extended by the configured timeout. For a `DEAUTH` packet, the same reverse lookup is used to find the active session, after which both the session index entry and the active session entry are removed.

After processing any knock packet, the XDP program returns `XDP_DROP`. This is intentional: the knock port is used only as a control channel and should not expose any service behavior to the host networking stack.

If a packet is not addressed to the knock port, the program checks whether the destination port is listed in the protected port list. Packets to unprotected ports are passed. Packets to protected ports are checked against `active_session_map` using the packet's full flow key. If no active session exists, if the session is marked for deletion, or if the session has expired, the packet is dropped. When an expired session is encountered, the program also removes the corresponding active session and session index entries. Only packets matching a valid, non-expired active session are passed to the host network stack.

B. Userspace Daemon and Loader

The userspace daemon is implemented in `knock_user`. In daemon mode, it accepts command-line parameters for the network interface, BPF object path, knock port, protected port list, session timeout, bind window, replay window, key source, duration, and BPF pin directory. The protected ports are parsed from a comma-separated list, while user keys can be loaded from a users file.

Before attaching the XDP program, the daemon validates the configuration. The session timeout and bind window must be non-zero, and the replay window must be large enough to cover the configured clock-skew window. The daemon also requires an interface name and at least one protected port. If neither a users file nor a fallback HMAC key is provided, the daemon exits instead of starting with an empty authentication configuration.

The lower-level loading logic is implemented in `xdp_loader`. The loader raises the memory-lock limit, opens the compiled BPF object, loads it through libbpf, locates `config_map`, and writes the runtime configuration into the map. It then reads both `CLOCK_MONOTONIC` and `CLOCK_REALTIME`, computes the realtime-to-monotonic offset, and stores that value in `time_offset_map`. This offset allows the XDP program to validate Unix timestamps in knock packets while using monotonic kernel time for runtime expiration.

The loader also preloads user keys into `user_key_map`. For each configured user, it initializes a `user_key_state` with the active HMAC key, key version one, and no previous-key grace period. After configuration and user insertion, the loader locates the `port_knock_xdp` program inside the BPF object and attaches it to the selected interface using XDP attach logic.

After a successful attach, the loader attempts to pin the important maps under the configured BPF filesystem directory. The pinned maps include the configuration map, pending authorization map, active session map, session index map, replay nonce map, source pressure map, statistics map, time offset map, debug snapshot map, and user key map. Pinning these maps allows separate administrative commands to inspect and modify runtime state without rebuilding or reattaching the XDP program.

When the daemon receives a termination signal or reaches its configured duration, it detaches the XDP program from the interface and closes the BPF object. This keeps the runtime lifecycle explicit: daemon startup loads, configures, pins, and attaches the program; daemon shutdown detaches it.

C. Knock Client

The knock client is implemented in `knock_client`. Its job is to construct signed knock packets in userspace and transmit them directly on the selected interface. The client builds a complete Ethernet, IPv4, TCP, and knock payload frame, fills the packet fields, computes checksums, and sends the frame through an `AF_PACKET` raw socket.

The client supports the same packet types handled by the XDP program: `auth`, `bind`, `renew`, and `deauth`. If the caller does not provide a timestamp, the client uses `CLOCK_REALTIME`. If the caller does not provide a nonce, the client generates a random nonce. If a session identifier is not provided for an authentication packet, the client generates a random session identifier. For authentication packets, the user identifier is encoded into the high bits of the session identifier so that the kernel-side program can derive the correct user key.

For `bind`, `renew`, and `deauth` packets, the client requires an explicit session identifier. For `bind`, it also requires a protected destination port to bind. The bind packet includes the intended source port and destination port in both the knock payload and the signature input. This ensures that the signed control message is tied to the flow that will later be authorized by the XDP program.

After filling the knock payload, the client computes the signature using the shared signature routine and the provided HMAC key. The signature input includes the timestamp, packet type, session identifier, nonce, and, for bind packets, the bind source and destination ports. The resulting signature words are stored in network byte order inside the knock packet. Finally, the full frame is sent using `sendto` on a raw packet socket bound to the selected interface.

D. Runtime User Management

Runtime user management is implemented as subcommands of `knock_user`. The available administrative operations are

register-user, rotate-user-key, revoke-user, and list-users. These commands operate on the pinned user_key_map, so they can update user state while the XDP program continues running.

The register-user command inserts a new user entry into user_key_map. It requires a user identifier and a 32-byte HMAC key encoded as 64 hexadecimal characters. If the user already exists, registration fails instead of overwriting the existing entry. A newly registered user has the provided key as its active key, key version one, and no previous-key grace period.

The rotate-user-key command updates an existing user entry. It first looks up the user in user_key_map. If the user exists, the current active key is copied into the previous-key field, the new key becomes the active key, the key version is incremented, and grace_until_ns is set using the configured grace period. This allows old clients to continue working for a limited time while new clients begin using the rotated key.

The revoke-user command removes a user entry from user_key_map. After revocation, new knock packets for that user can no longer be validated because the corresponding key state is absent. The list-users command iterates over user_key_map and prints the stored user identifiers, key versions, and grace expiration timestamps.

E. Session Renewal

Session renewal is implemented through the RENEW packet type. A normal active session has an expiration timestamp stored in its active_session_state. When the client sends a valid renew packet, the XDP program resolves the session identifier through session_index_map to find the corresponding flow key. It then looks up the active session in active_session_map. If the session still exists, is not marked for deletion, and has not expired, the program updates expires_at_ns by adding the configured timeout to the current time.

This renewal mechanism is useful for long-running protected connections such as SSH sessions. The original authentication and bind sequence creates a temporary flow-level authorization, while periodic renew packets keep the active session alive. If renew packets stop arriving, the session eventually expires and subsequent protected traffic for that flow is dropped. If the client sends a DEAUTH packet instead, the XDP program removes the session immediately by deleting both the reverse session index and the active flow entry.

The result is a bounded session model. A valid knock does not permanently open a service port, and even a successfully bound flow remains valid only while it is explicitly renewed before expiration.

VI. Evaluation

A. Experimental Setup

Testing was performed in a virtualized lab environment with two Fedora 43 virtual machines running under a Proxmox hypervisor. One VM acted as the protected host and ran the XDP stealth gate together with the protected SSH and HTTPS services. The second VM acted as the client and was used to send knock packets and attempt service connections.

B. Functional Tests

Functional testing focused on validating the expected access-control behavior. Direct connection attempts were tested before authentication, after a valid AUTH and BIND sequence, after session timeout, and after deauthentication or user revocation. The goal was to confirm that protected services were unreachable without valid authorization and reachable only while a valid flow-bound session existed.

C. Adversarial Security Reviews

In addition to functional testing, the codebase was reviewed using attacker-oriented security analysis. The purpose of this review was to identify implementation weaknesses that may not appear during normal correctness tests, including replay issues, unsafe packet parsing, inconsistent session state, incorrect key handling, and denial-of-service risks.

Multiple independent AI-assisted reviews were performed using GitHub Copilot Cloud Agents. The agents were not used as the primary implementation mechanism for the reviewed code; instead, they were used after implementation to provide an additional adversarial review perspective. Review prompts focused on how an attacker might bypass authentication, replay valid knock packets, exhaust eBPF maps, abuse pending or active session state, or trigger unsafe behavior in the XDP packet parser.

The review covered several areas of the implementation: bounds checking in the packet parser, signature verification inputs, user-key lookup and key rotation, replay nonce handling, pending authorization state, active flow-session binding, reverse session indexing, timeout cleanup, and source-pressure limits. Reported issues were manually inspected before being accepted, since AI-assisted review can produce false positives or suggest changes that are incompatible with eBPF verifier constraints.

The adversarial review process provided an additional layer of confidence in the prototype, but it should not be interpreted as a formal security proof. The system still requires fuzz testing, stress testing, and expert manual audit before being considered production-ready.

D. Security Behavior

The security behavior of the prototype follows the main state transitions in the XDP program: authentication, binding, renewal, deauthentication, replay rejection, timeout handling, and protected-traffic filtering. A valid AUTH packet does not directly open a protected service. It only creates temporary pending state. Access is granted only after a valid BIND packet converts that pending state into an active session bound to a specific flow tuple.

Invalid knock packets are dropped before they can modify authorization state. This includes malformed packets, invalid magic values, timestamps outside the allowed clock-skew window, unknown users, and signatures that do not match a valid

key. Replay protection is handled with a nonce map keyed by source address, nonce, packet type, and session identifier, preventing accepted knock packets from being reused within the replay window.

Both pending and active states are time limited. Pending authorizations expire after the bind window, and active sessions expire after the configured timeout unless renewed with a valid `RENEW` packet. A signed `DEAUTH` packet removes an active session by deleting both the active flow entry and the reverse session index. Administrative user revocation prevents that user from creating new valid knocks.

Overall, the system provides flow-bound access control rather than host-wide authorization. A successful knock sequence authorizes one specific protected flow, not all traffic from the same source address.

E. Performance Considerations

The implementation keeps expensive operations out of the common protected-data path. Signature verification is performed only for control packets: `AUTH`, `BIND`, `RENEW`, and `DEAUTH`. After a flow is authorized, normal protected traffic requires only header parsing, flow-key construction, and an `active_session_map` lookup.

Processing cost therefore depends on packet type. Non-IPv4 or non-TCP packets are passed after minimal parsing. Packets to unprotected TCP ports require only configuration and protected-port checks. Packets to protected ports require an active-session lookup. Knock packets are the most expensive because they require payload parsing, timestamp validation, user-key lookup, signature verification, replay checks, and map updates.

This design avoids per-packet cryptographic verification for ordinary service traffic. If every protected packet were signed, the XDP program would need to verify signatures and maintain replay or sequence state on the main data path, which would likely reduce throughput significantly. Instead, the system signs control packets and relies on protocols such as SSH or TLS for end-to-end traffic integrity.

VII. Limitations and Future Work

Although the proposed stealth gate reduces the visibility of protected services, it has several limitations. The first limitation is related to network address translation. The implementation binds active authorization to a full flow tuple, including source IP address, destination IP address, source port, destination port, and transport protocol. This is stronger than a simple source-IP allowlist, but NAT can still complicate the relationship between a user, a client machine, and the externally visible flow. Multiple clients may share the same public address, and source ports may be rewritten by the NAT device. As a result, the system should not be treated as a complete identity-management solution in NAT-heavy environments.

A second limitation is key compromise. The system relies on per-user shared secret keys for generating authenticated knock packets. If a user's key is stolen, an attacker can generate valid knock packets until the key is rotated or the user is revoked. The implementation supports key rotation and revocation, but it does not prevent compromise of the endpoint that stores the key. Future work could improve this model by using hardware-backed keys, asymmetric challenge-response authentication, or short-lived credentials issued by a central identity provider.

A third limitation is denial-of-service resistance. Because the XDP program runs early in the packet receive path, it can drop unauthorized packets before they reach the normal Linux networking stack or the protected service. However, this does not make the system immune to denial-of-service attacks. A large enough traffic flood can still saturate the network link, consume NIC resources, or pressure the XDP program itself. The current implementation includes source-pressure tracking for knock attempts and active sessions, but future work should evaluate stronger rate limiting, per-source penalties, and integration with upstream filtering.

Another design limitation is that the system authenticates control packets, not every packet in the protected data flow. This is intentional. In the current design, expensive signature verification is limited to `AUTH`, `BIND`, `RENEW`, and `DEAUTH` packets. Once a session is bound, ordinary protected traffic only requires packet parsing, flow-key construction, and a lookup in `active_session_map`. This keeps the XDP data path small and efficient.

Authenticating every protected traffic packet would significantly increase the processing cost. Each packet would require additional authentication metadata, signature verification, and replay or sequence-number tracking. If packet payloads were included in the signature, the verification cost would also scale with packet size. Depending on the exact design, per-packet signing could reduce throughput by one or two orders of magnitude compared with the current flow-bound model. It would also complicate TCP handling, because retransmitted, reordered, or duplicated segments would need to be handled without accidentally breaking valid connections or weakening replay protection.

For this reason, the proposed system leaves end-to-end traffic integrity and confidentiality to protocols such as SSH and TLS. The stealth gate is designed to decide whether a flow may reach the protected service, not to replace secure transport protocols. If per-packet cryptographic protection is required, future work should consider integration with existing secure transport mechanisms such as WireGuard, IPsec, TLS, or SSH rather than implementing custom per-packet signing inside the XDP fast path.

The current implementation is also limited in protocol scope. It focuses on IPv4 and TCP traffic. This is sufficient for protecting common services such as SSH and HTTPS, but it does not cover IPv6 or UDP-based services. Adding IPv6 support would require extending packet parsing, flow keys, replay handling, and configuration structures. UDP support would require a separate discussion of session semantics, because UDP does not provide the same connection structure as TCP.

Finally, the current implementation is a prototype rather than a complete production system. It lacks packaging, long-term observability, policy integration, and formal security analysis. Future work should include systemd integration, structured logging, Prometheus-compatible metrics, clearer operational tooling, automated cleanup of stale state, and a more rigorous threat evaluation. Formal verification is outside the scope of this project.

VIII. Conclusion

This work presented an eBPF/XDP-based stealth gate for reducing the exposure of protected network services. The system uses an XDP program attached to a network interface to inspect incoming IPv4/TCP packets before they reach the normal Linux network stack. Unauthorized packets targeting protected ports are dropped early, while authorized clients can obtain temporary access by sending signed knock packets.

The implementation uses a multi-stage authorization model. An authentication knock creates short-lived pending state indexed by the client source address and session identifier. A bind knock then converts this pending state into an active session bound to a full flow tuple. This design avoids opening protected ports globally for a client address and instead authorizes only the specific protected connection. Renewal and deauthentication packets allow the client to extend or remove active sessions without exposing the protected service directly.

The project also includes a userspace control plane for loading the XDP program, configuring protected ports, inserting user keys, and managing users through registration, key rotation, revocation, and listing commands. Runtime state is shared between userspace and the kernel through pinned eBPF maps, including configuration maps, user-key maps, pending authorization maps, active session maps, replay-protection maps, and debug counters.

Overall, the prototype demonstrates that eBPF and XDP can be used to implement a stealth-oriented access-control layer for host services. The approach does not replace SSH, TLS, or application-layer authentication, and it does not solve all denial-of-service or identity-management problems. However, it provides a practical foundation for hiding selected services from unauthenticated clients while preserving temporary, flow-bound access for authorized users. Future work should focus on IPv6 support, stronger authentication models, performance benchmarking, improved observability, and production-oriented deployment tooling.

References

[1] The Linux Kernel Documentation, "BPF Documentation." [Online]. Available: https://docs.kernel.org/bpf/

[2] The Linux Kernel Documentation, "eBPF Userspace API." [Online]. Available: https://docs.kernel.org/userspace-api/ebpf/index.html

[3] eBPF Docs, "Program Type BPF_PROG_TYPE_XDP." [Online]. Available: https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_XDP/

[4] R. deGraaf, J. Aycock, and M. Jacobson, "Improved port knocking with strong authentication," in *Proc. 21st Annual Computer Security Applications Conference*, 2005.

[5] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[6] National Institute of Standards and Technology, "The Keyed-Hash Message Authentication Code (HMAC)," FIPS PUB 198-1, July 2008. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/198/1/final

[7] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The eXpress Data Path: Fast programmable packet processing in the operating system kernel," in *Proc. 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2018, pp. 54–66.

[8] Cilium Documentation, "BPF and XDP Reference Guide." [Online]. Available: https://docs.cilium.io/en/stable/reference-guides/bpf/

[9] M. Kerrisk, "bpf(2): Linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/bpf.2.html

[10] libbpf Documentation, "LIBBPF API." [Online]. Available: https://libbpf.readthedocs.io/en/latest/api.html

[11] eBPF Docs, "Libbpf userspace function bpf_xdp_attach." [Online]. Available: https://docs.ebpf.io/ebpf-library/libbpf/userspace/bpf_xdp_attach/

[12] eBPF Docs, "Libbpf userspace function bpf_map_update_elem." [Online]. Available: https://docs.ebpf.io/ebpf-library/libbpf/userspace/bpf_map_update_elem/

[13] eBPF Docs, "Pinning." [Online]. Available: https://docs.ebpf.io/linux/concepts/pinning/

[14] bpftool Documentation, "bpftool-map(8): tool for inspection and simple manipulation of eBPF maps." [Online]. Available: https://man.archlinux.org/man/bpftool-map.8.en

[15] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015. [Online]. Available: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[16] D. Eastlake and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," RFC 6234, May 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6234

[17] I. Sen, "eBPF Secure Port Knock," GitHub repository. [Online]. Available: https://github.com/senivan/ebpf-secure-port-knock